

Design Situations: Understanding Rigor, Scope, Complexity, and the Nature of Systems

Highlights

- The need for rigor in a project
- Complexity and risk aversion as influences on the need for rigor
- Product delivery scope
- Consumer products vs. the enterprise systems

3.1 INTRODUCTION

As we said in [Section 2.4](#), each design situation requires you to make choices. Your job is to adopt and adapt UX lifecycle activities, methods, and techniques to specific project conditions. Most of the high-level process choices hinge on project parameters, such as rigor, scope, complexity, and whether you are designing a consumer product or an enterprise system.

3.2 RIGOR IN A UX METHOD OR PROCESS

Our use of the term *rigor* refers to the degree of formality, thoroughness, precision, and accuracy with which you perform all the steps of a UX design lifecycle activity, method, or technique. It is also about how meticulously you maintain and document completeness and purity of data collected, especially UX research data (see [Chapter 6](#)) and UX evaluation data (see [Chapters 22 and 23](#)).

Completeness. Completeness entails thoroughness of methods, which means covering every step in full and gathering all UX research and evaluation data available. This attention to detail helps designers touch all the bases and fill in all the blanks so that no functions or features are forgotten.

Purity. Purity means being as accurate as possible in your data. In particular, it means not allowing new spurious data to creep in. For example, for high UX research data purity, designer insights or conjectures should be tagged as such to distinguish them from actual user input. We refer to the methods for maintaining this completeness and purity of data as rigorous methods.

3.2.1 Complexity Influences the Need for Rigor
3.2.1.1 The system complexity space

One big reason you cannot define just one set of methods for designing all systems is that there is a spectrum of system and product types with a broad range of allowable risk and corresponding need for rigor in lifecycle activities and methods. In this and the next few sections, we look at some possibilities within this spectrum.

Fig. 3.1 shows a system complexity space defined by the dimensions of interaction complexity and domain complexity (described in the next two sections).

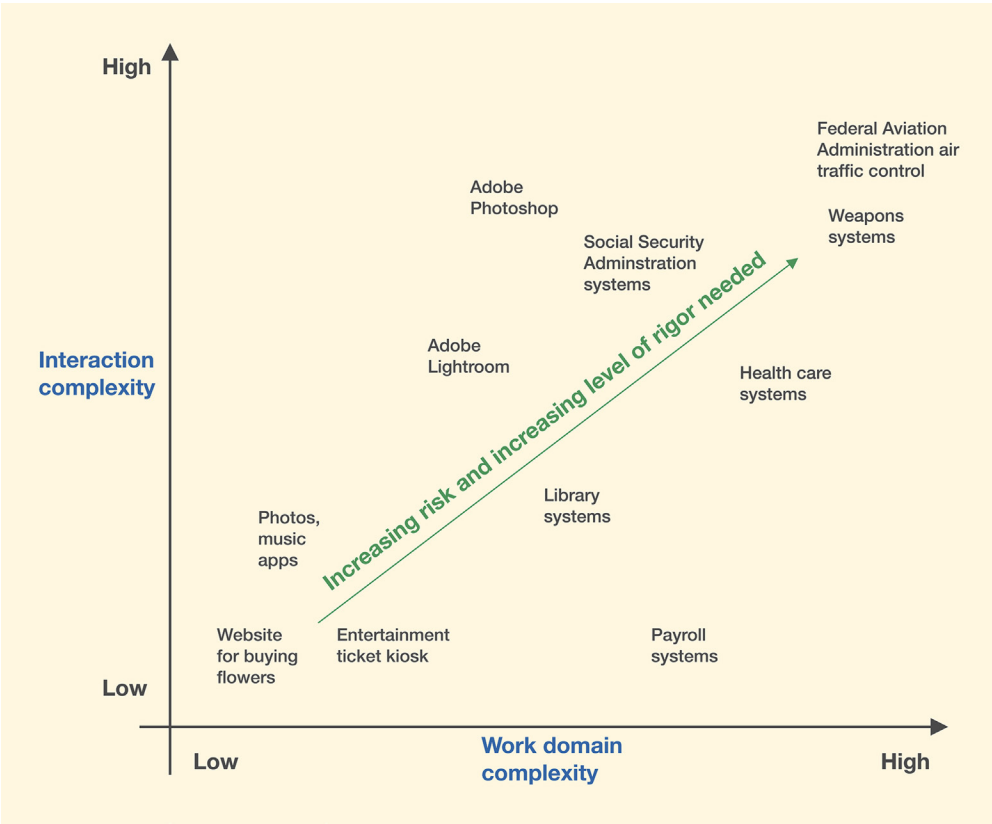


Fig. 3.1
The system complexity space: interaction complexity vs. domain complexity.

3.2.1.2 Interaction complexity

Interaction complexity, represented on the vertical axis, is about the intricacy or elaborateness of user actions, including the difficulty of cognitive actions necessary to accomplish tasks within the system. Low interaction complexity corresponds to systems that support smaller and easier tasks, such as ordering flowers from a website. High interaction complexity is usually associated with larger and more difficult tasks, often requiring special skills or training, such as manipulating a color image with Adobe Photoshop.

3.2.1.3 Domain complexity

On the horizontal axis, we show work¹-domain complexity, which is about the degree of intricacy and the technical, or possibly esoteric, nature of the corresponding field of work. Convolved and elaborate mechanisms for how parts of the system work and communicate within the ecology of the system contribute to domain complexity. Low work-domain complexity means that the way the system works within its ecology is relatively simple. Examples of low domain complexity include a website for buying flowers and a simple personal calendar management application.

User work in systems with high domain complexity is often mediated and collaborative, with numerous handoffs in a complicated workflow containing multiple dependencies and communication channels, along with compliance rules, regulations, and exceptions in the way work cases are handled. Examples of high work-domain complexity include systems for geological fault analysis for earthquake prediction, global weather forecasting, and complete health care systems.

3.2.1.4 The system complexity space quadrants

Simple interaction, simple work domain

The simplest quadrant is in the lower left corner of [Fig. 3.1](#), where both interaction and work domain are the least complex. This quadrant contains smaller websites, certain interactive applications, and many consumer products. Just because this is the simple-simple quadrant, however, does not mean that the products are overly simple; the products of this quadrant can still be sophisticated.

There is an abundance of relatively simple systems in the world. Some consumer software products are domain simple and interaction simple, at least

¹When we use the term *work*, we intend it to broadly signify anything people/users do while using products or systems (see Section 6.2).

relative to large systems of other types. Again, the website for ordering flowers is a good example of this quadrant. Interaction is very simple: just one main task involving a few choices and the job is done. Work-domain complexity is also relatively simple because it involves only one user at a time, and the workflow is almost trivial.

Although emotional impact factors do not apply to every system or product in this quadrant, emotional impact can be very important here, especially with respect to factors such as aesthetics or joy of use. A simple digital music player might be a good example of a consumer product in this quadrant. Because systems and consumer products in this quadrant have less complexity, their design and development often require less rigor.

Complex interaction, complex work domain

The most complex quadrant is in the upper right-hand corner of [Fig. 3.1](#), where both interaction and work domain are of high complexity.

Large and complex systems. An example is an air traffic control system used to decide the landing orders for an incoming airliner. An air traffic control system has enormous domain and interaction complexity, with workflow and collaboration among a large number of work roles and user types. Because of their sheer size and complexity, these systems are among the most difficult and expensive to design and develop.

The highest need for rigor. This quadrant is where you find the highest risks of design failure and, therefore, the highest need for rigor to manage risk. These project conditions include when:

- You have the greatest regulatory compliance requirements
- The importance of error avoidance in usage is highest (e.g., in mission-critical systems such as for air traffic control or for military weapons control)
- You cannot risk getting these things wrong, and the cost of failure is unacceptable
- You have a contractual obligation for high rigor

Resistance to innovation. In the large and complex systems of this quadrant, especially systems such as military weapons systems, long-term stability of designs is very important, and you are likely to encounter resistance to innovation. Radical designs are not always welcome; conformity can be thought more important. Users and operators, in some cases, commit operations to habit and perform tasks with learned behavior even if there are better ways. Work practice change must be approached with caution.

Enterprise systems. Among the kinds of systems one sees in the rest of this quadrant are the so-called enterprise systems, large information systems used within organizations. As Gillham (2014) puts it, “Most big businesses globally are locked into some kind of enterprise technology. Unfortunately, such systems are not only fiendishly difficult to install and maintain, but often equally challenging for the workforce to use. When the stakes are so high, why is the user experience of enterprise systems so bad?”

Complex interaction, simple work domain

In the upper left quadrant of [Fig. 3.1](#), one of the in-between quadrants, we see interaction-complex and domain-simple systems. It is typical of an interaction-complex system to have extensive functionality resulting in a large number and broad range of complex user tasks. The older-style digital watch (not a smartwatch) is a lightweight but good example. Its interaction complexity often stems from a large variety of modal settings using overloaded and unlabeled push buttons. The domain, however, is still simple, being mainly about knowing what time it is.

Attention in this quadrant is needed for interaction design—myriad tasks, screen layouts, user actions, and even metaphors. Rigorous formative evaluation may be needed for the consistency and usability of the conceptual design (see [Section 14.3](#)) and the detailed interaction design (see [Section 16.6](#)). The focus of modeling will be on tasks and perhaps the artifact model, but not much attention will be given to work roles, workflow, or most other models of [Chapters 8 and 9](#).

For simple work domains, regardless of interaction complexity, UX research is less likely to result in learning something new that can make a difference in informing design. So, less rigor will suffice in the UX research leading up to design creation. For design creation, however, complex interaction requires careful and systematic brainstorming, ideation, and sketching, plus iterative evaluation and refinement as well as attention to emotional impact factors.

Simple interaction, complex work domain

In the lower right quadrant of [Fig. 3.1](#), the other in-between quadrant, we see interaction-simple and domain-complex systems. In this quadrant, user tasks are relatively simple and easy to understand, so less attention to task descriptions is needed; interaction is relatively straightforward for users. The key effort for users in this quadrant is understanding the domain and its often esoteric work practice. Designers need rigorous UX research to focus

on conceptual design and easy-to-understand user models of how the system works. Tax preparation software for average households is a good example because the underlying domain is complex, but the data entry into forms can be simplified to a step-by-step process.

Sometimes systems for managing libraries, shown in the middle of the work-domain complexity scale near the bottom of [Fig. 3.1](#), fall into the simple interaction, complex work-domain quadrant. Typical library systems have low interaction complexity because the range of tasks and activities for any one user is fairly circumscribed and straightforward, and the complexity of any one user task is low. Therefore, for a small library system, for example, you might not need to exercise much rigor in the modeling of tasks.

However, a full library system has considerable domain complexity. The work practice of library systems can be specialized, and most UX designers will not be knowledgeable in this work domain. For example, specific training is needed to handle the surprisingly important and highly controlled small details in cataloging procedures. Therefore a rigorous approach to UX research may be warranted.

Gradations within the system complexity space

Some systems or products clearly fall into one quadrant or the other, but other projects can have fuzzy quadrant boundaries. Websites, in particular, can span multiple quadrants, depending on the nature of work they support and the breadth of their user communities. For example, compare the complexity of a very large ecommerce site with that of a small site for sharing photographs. Consumer products such as a printer or a camera can range over any level of domain complexity and might have medium interaction complexity.

Health care systems typically cross system complexity space quadrants. Some parts internal to a small doctor's office might be relatively simple, but large health care systems that integrate medical instrumentation, health record databases, and patient accounting have complex work domains. Similarly, machines in a patient's hospital room can have a fairly broad range of technical tasks and activities, giving them relatively high interaction complexity.

The health care domain is also saddled with more than its share of regulation, paperwork, and compliance issues, plus legal and ethical requirements—all of which lead to high work-domain complexity and a high need for rigor in UX lifecycle activities and methods.

3.2.2 Domain Familiarity Influences the Need for Rigor

Even if a domain is not complex in absolute terms, if it is unfamiliar to the UX designer, then it can seem complex, at least at first. Familiarity with the target domain is part of the designer's cognitive scaffolding for understanding the design problem. The way to achieve this domain familiarity is by initially using high-rigor UX methods.

In a former practice in the specialized domain of financial portfolio analytics, this author (Pyla) often encountered unfamiliar terminology. Or sometimes users from different portfolio management firms described different practices and philosophies on investment in different ways, making it difficult to maintain a unified understanding of the domain. During UX research data collection sessions with clients, users, and subject-matter experts, in order to maintain rigor, it was often important to capture metadata, such as who said what and where they worked, because those details were important for later analyses to contextualize what was said.

3.2.3 Risk Aversion Influences the Need for Rigor

Risk is the danger or possibility of things going wrong—features or requirements being missed or the result of not meeting users' needs, not accommodating legacy needs, or not complying with legal or safety constraints. An important goal-related factor in choosing the level of rigor in the UX design lifecycle activities and methods is aversion to risk. There are cases where not getting the UX design right creates high risks, often because of requirements for things such as legacy accommodation, legal constraints, or safety concerns.

The less there is tolerance for risks, the more there is need for rigor in the lifecycle activities, methods, and techniques. Of course, high rigor will add cost and time to the lifecycle process.

3.2.3.1 *The risk due to data loss*

One of the most significant kinds of rigor necessary to avert risk is the rigor with which you collect and maintain UX data. If your UX data get condensed, summarized, or otherwise abbreviated in the pursuit of speed and economy, you will suffer a loss of completeness of data. You can help reduce data loss by recording (audio or video) every detail of user interviews and observations, but it is tedious and costly to transcribe the recordings, and the result is a ton of text to wade through to separate the important stuff from all the noise and unimportant verbiage; the process is almost never worth it.

So, UX researchers often take notes to summarize the main points for themselves. Although this technically introduces a loss of completeness, it does not necessarily reduce the quality or usefulness of the data. Raw data need abstraction to remove irrelevant detail and to highlight what is important.

3.2.3.2 Risks associated with legal, safety, and compliance constraints

Systems such as air traffic control systems, health care and medical records systems, and financial systems have legal requirements with respect to public safety risks that must be taken very seriously. Designing a product for the financial industry requires a process that accounts for compliance to complex federal regulations built into the business process involved in using the product. Your lifecycle activities and methods need to incorporate sufficient rigor to ensure (and possibly even prove) that the system meets all compliance checks.

3.2.4 The Stage of Development Within Your Project Influences the Need for Rigor

The stage of development within your project is another determiner of the need for rigor. Regardless of method choices based on other project parameters, the appropriateness of a level of rigor and corresponding choices of UX methods and techniques for lifecycle activities will change as a project evolves through different stages of development. For example, early stages might demand a strong focus on rigorous UX research to obtain the most user, usage, and domain knowledge upfront, but there might be very little emphasis on rigorous evaluation in early stages. Spending large resources on early evaluation could be a waste because the design is still evolving. So, for early stages, it might be better to use lightweight, rapid, and less rigorous evaluation methods (see [Chapter 23](#)). As an example, to evaluate an early conceptual design, you might choose a quick design walkthrough (see [Section 23.3](#)).

In later stages of evaluation, to refine a now-stable design, you might move to UX inspection (see [Section 23.4](#)) of a low-fidelity prototype and eventually to the more rigorous lab-based evaluation (see [Chapter 22](#)) using a high-fidelity prototype, increasing the amount and quality of data you retain and, consequently, increasing rigor.

3.2.5 Project Resources Influence the Ability to Provide Rigor

High-rigor costs money and takes time. Slim budgets and short schedules are practical realities that restrict the level of rigor you are able to provide. Another important kind of resource is person power. How many people do you have, what project team roles can they fill, and what UX skills, experience, and training do they bring to the project?

UX professionals with extensive experience and maturity are likely to need less of the formal aspects of rigorous methods, such as thorough UX research or detailed UX evaluation goals and targets. For these experienced UX professionals, following the process in detail does not add much to what they can accomplish using their already internalized knowledge and honed intuition.

3.2.6 UX Team Experience as Influence on the Need for Rigor

Highly experienced teams will need less rigor in their process because they have internalized the process, and more rigor will not add value. If your UX team is inexperienced with the domain and/or with the UX process, obviously you have to use more rigor and proceed slowly and carefully until you feel confident in your UX skills.

3.3 BEING RAPID IN LIFECYCLE ACTIVITIES, METHODS, AND TECHNIQUES

While some methods are inherently more rigorous than others (e.g., lab-based evaluation vs. inspection methods), it is important to note that any method or technique can be performed across a range of rigor. Applying an evaluation method with high rigor can achieve more complete and more accurate results, but it will take more time and is more costly. Similarly, by taking shortcuts, you can usually increase speed and reduce costs of almost any UX evaluation method, but it will be at the price of reduced rigor.

3.3.1 Not Every Project Needs Rigorous UX Methods

For many projects, certainly those involving consumer products and often those involving enterprise systems, high rigor is not necessary, is not worth the cost, or simply is not possible given limited project resources.

3.3.2 Over Time Our Need for Rigor in UX Design Has Diminished

As another factor, UX practice has matured, and we have gotten better at our craft. As a result, we do not need to be as rigorous and thorough in most common UX situations as we previously were. Less rigorous UX methods and techniques have evolved in the literature and practice that are faster and less expensive, but they still provide good results from your effort and resources. Now the standard method for each lifecycle activity is a combination of a practical middle-of-the-road rigor and some obvious ways to be efficient.

3.3.3 Rapid Methods Are a Natural Result

When higher rigor is not required, you can reduce lifecycle cost and time by abbreviating the standard rigorous methods and techniques, taking shortcuts, skipping unnecessary steps, and maintaining only the most important data. Or you can use rapid alternative methods that are inherently less rigorous.

3.3.4 Rapidness Principle: Work as Rapidly as You Can

The principle underlying all choices of lifecycle activities, methods, and techniques is: Go as fast as you can, subject to constraints imposed by project goals and the need for rigor. The agile approach to both UX and software engineering has proven that it is a misconception to think slow and deliberate work will produce a better product and that you will not leave any problems to fix later (Adler, 2011). Of course, we know that is never possible, so we work more rapidly, with the knowledge that we will always have the opportunity to fix problems in time.

Being economical and lightweight is now a way of life for an agile designer. Even when you need a rigorous approach, you should do what comes naturally in making smart choices and not wasting time or resources needed to do things that will not contribute to your final design. For example, an otherwise rigorous approach could be done without one or more types of data models (see [Chapters 8 and 9](#)) if they are non-essential to the project.

3.4 SCOPE OF DELIVERY

Our use of the term *scope* refers to how the target system or product is chunked in each iteration or sprint for delivery in agile implementation. The UX role is to deliver designs to the software engineering people in chunks of a given size, and their role consumes the chunks of UX design as specifications

for code, which they implement and deliver as chunks of functional software to clients and users.

In a large scope, chunks are composed of multiple features or even large portions of the system. In a small scope, synonymous with agility, chunks are usually comprised of one or few features at a time. We, the UX designers, will deliver our designs in chunks of a small scope within an agile UX process (see [Chapters 4 and 26](#)). However, some early UX design work (e.g., to establish the conceptual design) may require a larger scope. A large scope offers a nice structure for describing the UX lifecycle activities in this book.

3.4.1 Example: Large and Small Scope in Building a House

Here we use a familiar example, designing and building a house, to illustrate the tradeoffs between large and small scope in simple terms. Here is how scope plays out in terms of how you approach designing and building a house:

- Large scope: Design the whole house first, and build it all before delivering it to the client.
- Small scope: Design one room first (e.g., the kitchen or only the cooking area of the kitchen), build it, and deliver it to the client. Ask the client to view this room or feature and provide feedback, which you use to adjust the design and construction. Follow this pattern with each room of the house in increments until the whole house is completed.

For practical reasons, of course, the whole house must be designed before construction begins, with the exception of a few small adjustments that can be made later. It would be difficult and very expensive for architects and builders to work together in a small-scope agile manner, but it is intriguing to think of what amazing structures could be built if there were enough resources to carry out a truly agile house-building process.

3.5 CONSUMER PRODUCTS VS. ENTERPRISE SYSTEMS

Another factor that influences your UX design approach is whether we are dealing with a consumer product or an enterprise system. Obviously, it is inappropriate to use the same method to design a music application as you would, say, an enormous corporate resource management system.

3.5.1 Consumer Products

We use the term *consumer products* to describe personal objects, such as music devices or software apps, that a user buys for private use.

Consumer products can still involve multiple users (e.g., people sharing music), multiple activities (e.g., buying music, organizing music), and multiple systems (e.g., a network of personal phones, tablets, and entertainment devices connected to the cloud).

3.5.2 Enterprise System

We use the term *enterprise system* to refer to systems where the work practice involves multiple users and actors spanning multiple activities, usually in an organizational setting. Think, for example, of a corporate information management system. The goal of work practice in this context is to further the business goals of that organization. Organizations contain numerous and often widely different system usage roles, each contributing to part of the overall work that gets accomplished.

3.6 SEGUE

In this chapter, we discussed the ideas of scope and rigor and how they influence project development choices for building systems with different interaction and domain complexity. In the next chapter, we introduce a funnel model of agile UX design, delving deeper into the scope dimension and discuss UX processes that are resilient and responsive to change.